SIMATS ENGINEERING ASSESSMENT
TOOL 2

NAME: LAHARI REDDY **REG NO** : 192411164
COURSE NAME: INTERNET PROGRAMMING **COURSE CODE:** CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 93**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

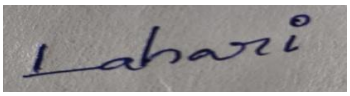
Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I LAHARI REDDY (192411164) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	MARKS
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	23
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	23
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	19
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	19
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	9

1. HTTP Response Analysis

Q1. Identify any incorrect or invalid fields in the header.

The incorrect field is:

- Content-Length: -150 – Invalid because Content-Length cannot be negative.

- Date: Mon, 01 Apr 2026 10:00:00 GMT – The weekday is incorrect; 1 April 2026 is a Wednesday, not Monday.

Q2. Analyze the issue with Content-Length and explain its impact.

Content-Length specifies the size of the response body in bytes. A negative value is invalid and may cause the response to be rejected or handled as malformed by HTTP software.

Q3. Determine whether the status code matches the response correctness.

200 OK indicates that the request was successful. However, the invalid Content-Length makes the response header malformed. Therefore, the status code alone does not guarantee that the entire response is correct.

Q4. Suggest corrections for the identified errors.

Corrected header:

HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

Here, 150 represents an example body size of 150 bytes. The actual value must match the response body length.

2. DOM Structure Analysis

Q1. Identify missing or improperly closed elements.

- The element is missing its closing tag.
- The <title> element is correctly closed.
- The <head>, <body>, and <html> elements are correctly closed.

Q2. Analyze how the browser will interpret this structure.

Browsers use HTML parsing rules to recover from malformed markup. When the parser encounters , it will implicitly close the open paragraph in this example, producing a corrected DOM structure.

Q3. Determine the impact on rendering and layout.

- The browser may still display the page.
- Paragraph spacing may differ from the intended layout.
- CSS styling may be affected by the resulting DOM structure.
- Malformed HTML can create accessibility and maintenance problems.

Q4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>TestPage</title>
```

```
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

The corrected code includes the missing tag and the language and character-encoding declarations.

3. GET vs POST in Login System

Q1. Interpret the differences in data transmission between GET and POST.

- GET: Sends data as URL query parameters.
- POST: Sends data in the HTTP request body.
- GET is generally used to retrieve resources, whereas POST is commonly used to submit data.

Q2. Analyze which method is more suitable for login and why.

POST is more suitable for login because credentials are sent in the request body rather than as URL parameters. However, POST alone does not encrypt the data; HTTPS is essential.

Q3. Identify security risks in using GET for authentication.

1. Passwords may appear in browser history.
2. URLs may be stored in server logs.
3. URLs can be copied or shared accidentally.
4. Credentials may leak through monitoring or analytics systems.

Q4. Suggest best practices for secure login implementation.

1. Use POST over HTTPS.
2. Validate credentials on the server.
3. Store passwords using secure hashing algorithms such as Argon2id or bcrypt.
4. Use secure session cookies.
5. Implement rate limiting to prevent repeated login attempts.
6. Never store passwords in plain text.

4. HTML Code Inspection

Q1. Identify missing closing tags.

- The <h1> element is missing </h1>.
- The element is missing .

The <img> element does not require a closing tag because it is a void element in HTML.

Q2. Analyze issues with the <img> tag.

The image tag is missing the alt attribute, which provides alternative text for users who cannot see the image. The image file must also exist at the specified path.

Q3. Determine how browsers will handle these errors.

Browsers generally attempt to repair malformed HTML automatically. They may implicitly close the heading and paragraph elements, allowing the page to render, but the result may not match the intended structure.

Q4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Sample</title>
</head>
<body>
   <h1>Welcome</h1>
   <p>This is a paragraph.</p>
   <img src="image.jpg" alt="Sample image">
</body>
</html>
```

5. Browser-Server Communication Flow

Q1. Interpret each step in the communication flow.

1. User enters a URL: The browser receives the website address.
2. DNS lookup: DNS translates the domain name into an IP address.
3. HTTP request is sent: The browser establishes the required connection and sends an HTTP request.
4. Server processes the request: The server locates or generates the requested resource.
5. Response is returned: The server sends an HTTP status code, headers, and possibly a response body.
6. Browser renders the page: The browser parses HTML, CSS, and JavaScript and displays the webpage.

Q2. Identify where delays can occur.

- DNS resolution.
- Establishing the TCP connection and TLS handshake when applicable.
- Slow network transmission.
- Server processing and database queries.
- Downloading large images, CSS, JavaScript, or other resources.

- Browser parsing, JavaScript execution, and rendering.

Q3. Analyze the role of DNS in the process.

DNS (Domain Name System) translates a domain name, such as example.com, into an IP address that the browser can use to contact the server. DNS caching reduces repeated lookups and can improve loading speed.

Q4. Suggest optimizations to improve performance.

1. Use DNS caching and reliable DNS services.
2. Enable HTTP caching for reusable resources.
3. Compress responses using Brotli or gzip.
4. Optimize images and minimize unnecessary JavaScript and CSS.
5. Use a Content Delivery Network (CDN).
6. Optimize server code and database queries.
7. Use HTTP/2 or HTTP/3 where supported.
8. Reduce unnecessary network requests.

Conclusion: Correct HTTP headers, valid HTML, secure form submission, and optimized browser-server communication improve web application reliability, security, accessibility, and performance.